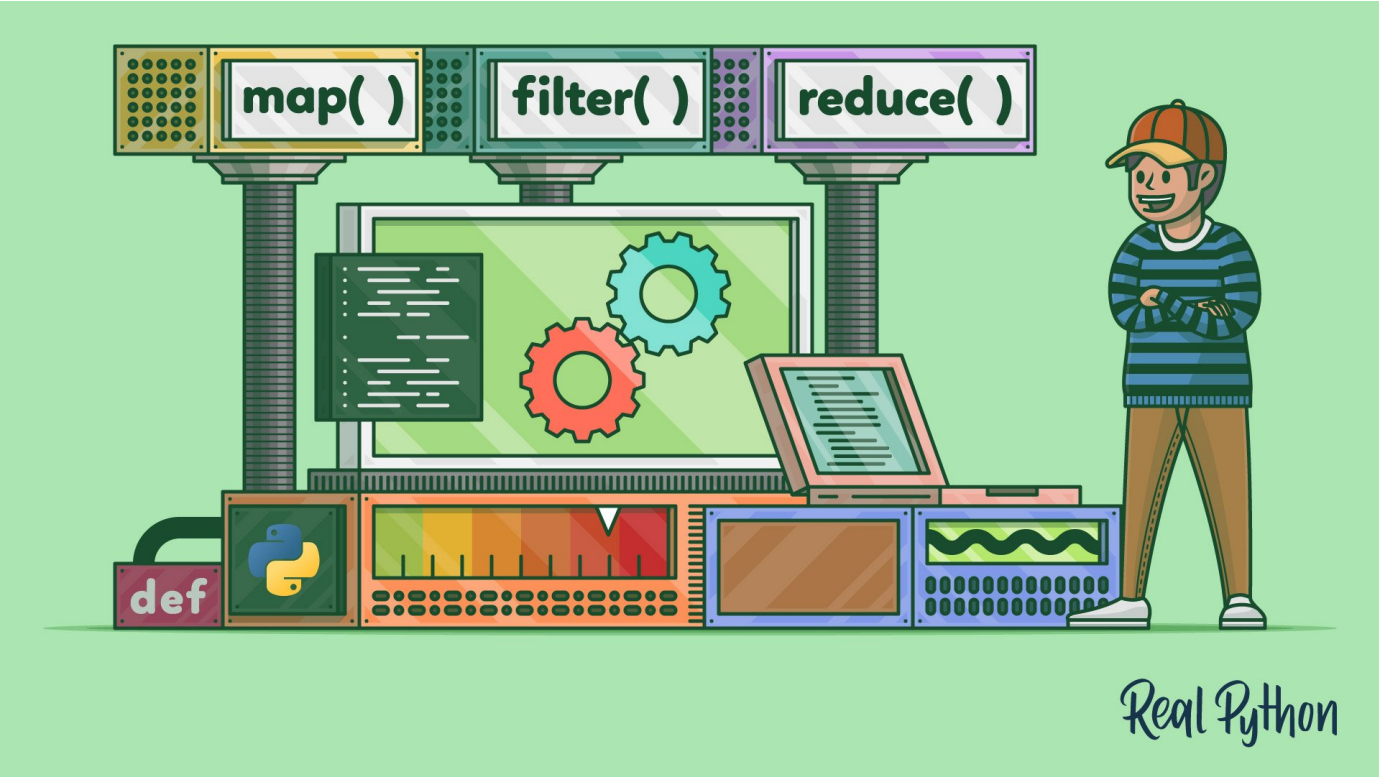
USING FUNCTIONAL PROGRAMMING IN PYTHON



YOU WILL LEARN ABOUT:

1. Functional programming
2. Functions as objects
3. `map()`
4. `filter()`
5. `reduce()`

VERSIONS

Note:

- Sample code tested using:
  - Python 3.14

OVERVIEW

- Functional programming is an approach that composing functions together
- Python doesn’t require a functional programming approach but does support it
- Functions are objects
  - Assign values to them
  - Pass them as references
  - Return references to them
- Python comes with a selection of functional programming tools

NEXT UP...

Functional programming approach

TABLE OF CONTENTS

1. Introduction
2. **Functional Programming**
3. Functions as Objects
4. Lambdas
5. `map()`
6. `filter()`
7. `reduce()`
8. Summary

PROGRAMMING APPROACHES

```
# Procedural
x =
y =
prin
```

```
# Object-Oriented
class
    d
    d
```

```
# Functional
def work(values, callback_fn):
    total = sum(values)
    callback_fn(total)
```

FUNCTIONAL PROGRAMMING

- Functional programming typically needs:
  - Functions being able to be passed as arguments (not just executed)
  - Functions being able to return a function reference
- In Python functions are first-class citizens
  - Everything in Python is an object, even functions
  - Share basic properties of strings, integers, etc

WHY FUNCTIONAL PROGRAMMING?

- Pure functional programming avoids side effects
- Compose functions together
- As state tends to be passed (no side effects) it is self contained
  - Transparent behavior
  - Parallelizable

NEXT UP...

Functions as objects

TABLE OF CONTENTS

1. Introduction
2. Functional Programming
3. **Functions as Objects**
4. Lambdas
5. `map()`
6. `filter()`
7. `reduce()`
8. Summary

NEXT UP...

Lambdas

TABLE OF CONTENTS

1. Introduction
2. Functional Programming
3. Functions as Objects
4. **Lambdas**
5. `map()`
6. `filter()`
7. `reduce()`
8. Summary

LAMBDAS

- Functions have overhead in Python
- Sometimes you want something lighter weight
- Lambdas are:
  - Anonymous
  - Can be used inline
  - Restricted to a single statement

NEXT UP...

`map()`

TABLE OF CONTENTS

1. Introduction
2. Functional Programming
3. Functions as Objects
4. Lambdas
5. `map()`
6. `filter()`
7. `reduce()`
8. Summary

PYTHON'S `map()`

```
map(callable_ref, iterable)
```

- Apply a function to each item in an iterable

NEXT UP...

`filter()`

TABLE OF CONTENTS

1. Introduction
2. Functional Programming
3. Functions as Objects
4. Lambdas
5. `map()`
6. `filter()`
7. `reduce()`
8. Summary

PYTHON'S `filter()`

```
filter(callable_ref, iterable)
```

- Filter out results from an iterable

NEXT UP...

`reduce()`

TABLE OF CONTENTS

1. Introduction
2. Functional Programming
3. Functions as Objects
4. Lambdas
5. `map()`
6. `filter()`
7. `reduce()`
8. Summary

PYTHON'S `reduce()`

```
reduce(callable_ref, iterable)
⋮
output = iterable[0]
for item in iterable[1:]:
    output = callable_ref(output, item)
```

- Compose results based on previous result and the next item in the iterable

NEXT UP...

Summary

TABLE OF CONTENTS

1. Introduction
2. Functional Programming
3. Functions as Objects
4. Lambdas
5. `map()`
6. `filter()`
7. `reduce()`
8. **Summary**

SUMMARY

- Functional programming chains function calls together
- Requires the use of functions as both data and invocation targets
- Functions in Python are objects
- Python lambdas are anonymous, single statement functions
- Three common functional programming utilities are:
  - `map()`
  - `filter()`
  - `reduce()`

MORE AT REAL PYTHON

- Using Python Lambda Functions
  - https://realpython.com/courses/python-lambda-functions/
  - https://realpython.com/python-lambda/ (tutorial)
- `map()`, `filter()`, and `reduce()`
  - https://realpython.com/python-map-function/ (tutorial)
  - https://realpython.com/python-filter-function/ (tutorial)
  - https://realpython.com/python-reduce-function/ (tutorial)
- Speed Up Python With Concurrency
  - https://realpython.com/courses/speed-python-concurrency/
  - https://realpython.com/python-concurrency/ (tutorial)

Thanks!